

PLAN DE PRUEBAS DE SOFTWARE

Proyecto: Desarrollo Seguro

Aplicación Web PHP con Arquitectura MVC

Proyecto:	Desarrollo Seguro - Sistema de Inventario
Versión:	1.0
Fecha:	Abril - Mayo 2026
Herramienta:	PHPUnit 9.6.34
Lenguaje:	PHP 8.2.12
Base de Datos:	MySQL - db_inventory (Puerto 3307)
Total de Pruebas:	45 pruebas - 47 assertions - 100% PASS

1. Introducción

El presente documento describe el Plan de Pruebas para el proyecto 'Desarrollo Seguro', una aplicación web desarrollada en PHP con arquitectura MVC (Modelo-Vista-Controlador) que gestiona un sistema de inventario con control de acceso por roles.

Las pruebas fueron ejecutadas utilizando PHPUnit 9.6.34, herramienta estándar para pruebas unitarias y de integración en PHP. Este plan sigue los lineamientos del estándar IEEE 829 y la metodología descrita en el documento 'Las Pruebas en el Desarrollo de Software' (Campos, 2015).

Se implementaron tres niveles de prueba: unitarias, de integración y de validación/seguridad, cubriendo tanto el back-end como la base de datos del sistema.

2. Objetivo

Verificar que los componentes del proyecto 'Desarrollo Seguro' cumplen con los requerimientos funcionales definidos, garantizando la calidad del software según el estándar ISO 9126 mediante:

- Pruebas Unitarias: validar el comportamiento individual de cada método de la clase User
- Pruebas de Integración: verificar la interacción correcta entre los componentes y la base de datos
- Pruebas de Validación: verificar partición equivalente, valores límite y seguridad (SQL Injection, XSS)

3. Alcance

3.1 Componentes Probados

- models/User.php — Clase principal con toda la lógica de negocio
- models/DataBase.php — Clase de conexión a la base de datos
- models/Validator.php — Clase de validación y seguridad creada para las pruebas
- Base de datos db_inventory — Tablas ROLES y USERS

3.2 Niveles de Seguridad Probados

Nivel	Qué se probó	Cómo
Back-end	Validaciones de campos (nombre, email, ID, estado)	ValidationTest.php
Base de Datos	CRUD, conexión, integridad de datos	IntegrationTest.php
Seguridad	Inyección SQL y ataques XSS	ValidationTest.php

4. Estrategia de Pruebas

4.1 Tipos de Pruebas Aplicadas

Pruebas Unitarias (UserTest.php): Se probaron los métodos setters y getters de la clase User de forma aislada, sin dependencia de la base de datos.

Pruebas de Integración (IntegrationTest.php): Se probó la interacción real entre la aplicación y la base de datos MySQL, verificando operaciones CRUD completas.

Pruebas de Validación y Seguridad (ValidationTest.php): Se aplicó partición equivalente, valores límite y pruebas de seguridad contra SQL Injection y XSS.

4.2 Técnicas Aplicadas

- Partición Equivalente: clases válidas e inválidas para cada campo
- Valores Límite: exactamente 15 caracteres (válido) y 16 caracteres (inválido)
- Caja Negra: validando entradas y salidas sin ver la implementación interna
- Integración Ascendente (Bottom-Up): primero unitarias, luego integración

4.3 Herramientas Utilizadas

- PHPUnit 9.6.34 — Framework de pruebas automatizadas para PHP
- Composer 2.9.7 — Gestor de dependencias PHP
- XAMPP — Servidor local Apache + MySQL
- PHP 8.2.12 — Lenguaje de programación

- MySQL (puerto 3307) — Base de datos
- phpMyAdmin — Administrador visual de base de datos

5. Base de Datos - db_inventory

La base de datos db_inventory contiene 7 tablas que soportan el sistema. Las tablas principales probadas son ROLES y USERS:

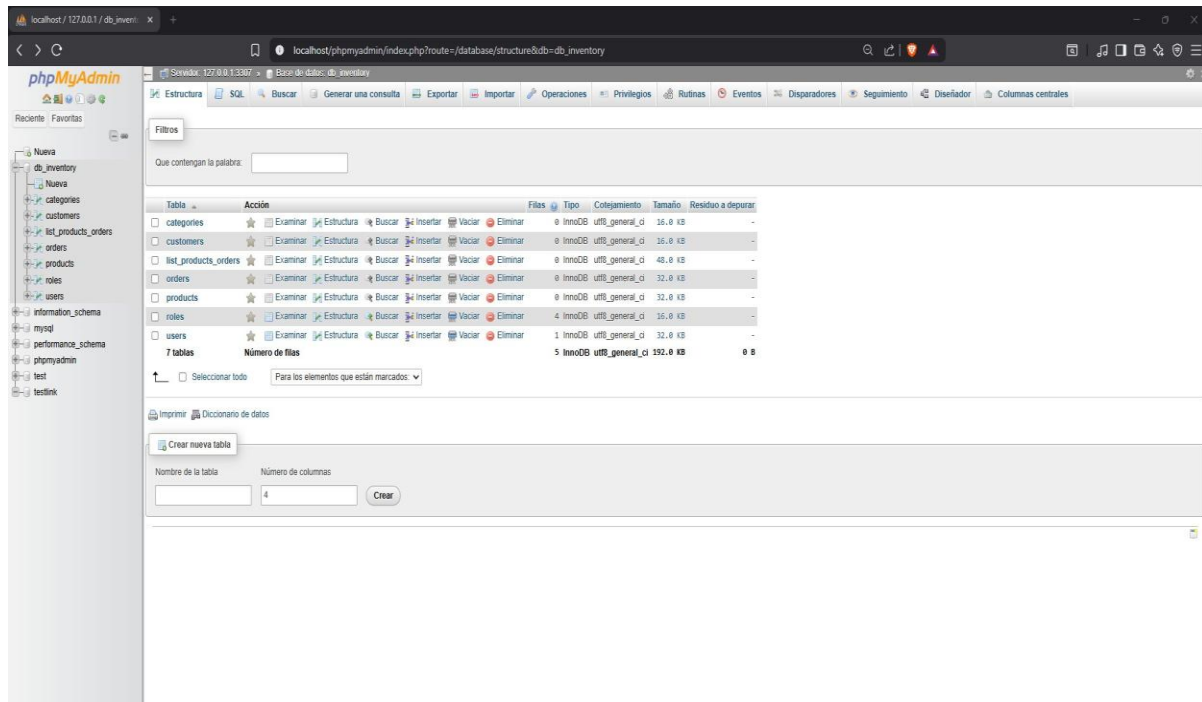


Tabla	Acción	Files	Tipo	Colección	Tamaño	Residuo a depurar
☐ categorias		0	InnoDB	utf8_general_ci	16.0 KB	-
☐ customers		0	InnoDB	utf8_general_ci	16.0 KB	-
☐ list_products_orders		0	InnoDB	utf8_general_ci	48.0 KB	-
☐ orders		0	InnoDB	utf8_general_ci	32.0 KB	-
☐ products		0	InnoDB	utf8_general_ci	32.0 KB	-
☐ roles		4	InnoDB	utf8_general_ci	16.0 KB	-
☐ users		1	InnoDB	utf8_general_ci	32.0 KB	-
7 tablas	Número de filas	5	InnoDB	utf8_general_ci	192.0 KB	0 B

Crear nueva tabla

Nombre de la tabla: Número de columnas:

Figura 1. Estructura de la base de datos db_inventory en phpMyAdmin

6. Pruebas Unitarias (UserTest.php)

Las pruebas unitarias validan cada método de la clase User de forma aislada. Se verifican los 8 pares de setter/getter:

ID	Test	Entrada	Esperado	Resultado
UT-01	testSetAndGetUserName	setUserName('Juan')	getUserName()='Juan'	PASS ✓
UT-02	testSetAndGetUserLastName	setUserLastName('Perez')	getUserLastName()='Perez'	PASS ✓
UT-03	testSetAndGetUserEmail	setUserEmail('juan@correo.com')	getUserEmail()='email'	PASS ✓
UT-04	testSetAndGetUserPass	setUserPass('123456')	getUserPass()='123456'	PASS ✓
UT-05	testSetAndGetUserState	setUserState(1)	getUserState()='1'	PASS ✓
UT-06	testSetAndGetRolCode	setRolCode('admin')	getRolCode()='admin'	PASS ✓
UT-07	testSetAndGetRolName	setRolName('Administrador')	getRolName()='Administrador'	PASS ✓
UT-08	testSetAndGetUserId	setUserId('123456789')	getUserId()='123456789'	PASS ✓

```
PS C:\xampp\htdocs\desarrollo_seguro> vendor\bin\phpunit.bat tests\UserTest.php
PHPUnit 9.6.34 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\desarrollo_seguro\phpunit.xml

.....                                         8 / 8 (100%)

Time: 00:00.035, Memory: 6.00 MB

OK (8 tests, 8 assertions)
PS C:\xampp\htdocs\desarrollo_seguro> 
```

Figura 2. Resultado de ejecución de pruebas unitarias - 8/8 PASS

7. Pruebas de Integración (IntegrationTest.php)

Las pruebas de integración verifican que los componentes del sistema funcionan correctamente en conjunto, incluyendo la interacción real con la base de datos MySQL:

ID	Test	Componentes	Resultado
IT-01	testDatabaseConnection	PHP + MySQL	PASS ✓ - Conexión exitosa
IT-02	testReadRoles	User.php + DataBase.php + ROLES	PASS ✓ - 4 roles encontrados
IT-03	testReadUsers	User.php + DataBase.php + USERS	PASS ✓ - Usuarios encontrados
IT-04	testCreateAndDeleteRol	create_rol() + delete_rol()	PASS ✓ - CRUD completo exitoso
IT-05	testAdminUserExists	read_users() + USERS	PASS ✓ - Admin verificado en BD

```
PS C:\xampp\htdocs\desarrollo seguro> vendor\bin\phpunit.bat tests\IntegrationTest.php
PHPUnit 9.6.34 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.12
Configuration: C:\xampp\htdocs\desarrollo seguro\phpunit.xml

.....                                     5 / 5 (100%)

Time: 00:00.101, Memory: 6.00 MB

OK (5 tests, 7 assertions)
```

Figura 3. Resultado de ejecución de pruebas de integración - 5/5 PASS

8. Pruebas de Validación y Seguridad (ValidationTest.php)

Se aplicó la técnica de partición equivalente y valores límite sobre todos los campos del sistema, además de pruebas de seguridad contra ataques comunes:

8.1 Partición Equivalente - Nombre y Apellido

ID	Clase de Equivalencia	Valor de Prueba	Resultado
VT-01	Válida: nombre con letras	'Juan'	PASS ✓ - Aceptado
VT-02	Inválida: nombre con números	'Juan123'	PASS ✓ - Rechazado
VT-03	Inválida: caracteres especiales	'J@an#'	PASS ✓ - Rechazado
VT-04	Límite: exactamente 15 caracteres	'Juancarlosalber'	PASS ✓ - Aceptado
VT-05	Límite: 16 caracteres (excede)	'Juancarlosalbero'	PASS ✓ - Rechazado
VT-06	Inválida: campo vacío	"	PASS ✓ - Rechazado

8.2 Partición Equivalente - Email

ID	Clase de Equivalencia	Valor de Prueba	Resultado
VT-11	Válida: email correcto	'juan@correo.com'	PASS ✓ - Aceptado
VT-12	Inválida: sin @	'juancorreo.com'	PASS ✓ - Rechazado
VT-13	Inválida: sin dominio	'juan@'	PASS ✓ - Rechazado
VT-14	Inválida: solo texto	'juancorreo'	PASS ✓ - Rechazado

8.3 Seguridad - SQL Injection y XSS

ID	Tipo de Ataque	Valor de Prueba	Resultado
ST-01	SQL Injection básico	"" OR '1'='1"	PASS ✓ - Bloqueado
ST-02	SQL DROP TABLE	'DROP TABLE USERS'	PASS ✓ - Bloqueado
ST-03	Texto normal (válido)	'Juan Perez'	PASS ✓ - Permitido
ST-04	XSS con script	'<script>alert(xss)</script>'	PASS ✓ - Bloqueado

ST-05	XSS con img tag	''	PASS ✓ - Bloqueado
-------	-----------------	--------------------------	--------------------

```

PS C:\xampp\htdocs\desarrollo seguro> vendor\bin\phpunit.bat tests\ValidationTest.php
PHPUnit 9.6.34 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.12
Configuration: C:\xampp\htdocs\desarrollo seguro\phpunit.xml

.....                                     32 / 32 (100%)

Time: 00:00.009, Memory: 6.00 MB

OK (32 tests, 32 assertions)

```

Figura 4. Resultado de pruebas de validación y seguridad - 32/32 PASS

9. Resumen Final de Ejecución

Archivo	Tipo	Pruebas	Assertions	Resultado
UserTest.php	Unitarias	8	8	100% ✓
IntegrationTest.php	Integración	5	7	100% ✓
ValidationTest.php	Validación/Seg.	32	32	100% ✓
TOTAL	3 archivos	45	47	45/45 ✓

10. Observaciones de Seguridad

ID	Descripción	Ubicación	Recomendación
OB-01	Inconsistencia en hashing: sha1() vs password_hash()	User.php: create_user() y update_user()	Unificar usando password_hash()
OB-02	Credenciales de BD expuestas en código	DataBase.php: root sin contraseña	Usar variables de entorno (.env)
OB-03	Controlador cargado dinámicamente sin whitelist	index.php: parámetro ?c=	Validar con lista de controladores permitidos

11. Conclusiones

- Se ejecutaron exitosamente 45 pruebas en total con un 100% de éxito: 8 unitarias, 5 de integración y 32 de validación/seguridad.
- Las pruebas unitarias confirmaron que todos los métodos setter y getter de la clase User funcionan correctamente de forma aislada.
- Las pruebas de integración verificaron que la conexión, consultas CRUD e integridad de datos con MySQL funcionan correctamente.
- La partición equivalente cubrió clases válidas e inválidas para todos los campos: nombre, apellido, email, contraseña, ID y estado.
- Los valores límite fueron probados verificando el límite de 15 caracteres en nombre y apellido.
- Las pruebas de seguridad detectaron correctamente intentos de SQL Injection y ataques XSS.
- Se identificaron 3 observaciones de seguridad que deben ser atendidas antes de pasar el sistema a producción.
- Las pruebas automatizadas con PHPUnit permiten ejecutar los 45 casos con un solo comando, garantizando eficiencia y repetibilidad.

Recomendación final: El sistema puede continuar con su desarrollo atendiendo las observaciones de seguridad identificadas. Las pruebas demuestran que la arquitectura MVC está correctamente implementada.